SQL Query Builders and Interactive Query Interfaces: A Comprehensive Overview

SQL query builders and interactive query interfaces represent a significant evolution in how users interact with databases, making data querying more accessible to both technical and non-technical users. These tools abstract away the complexity of raw SQL while maintaining powerful querying capabilities.

What Are SQL Query Builders?

An SQL query builder is a programming library or visual software tool that simplifies the construction of SQL queries. These tools are database-aware and provide users with hints, procedures, and structured methods to help them write queries without needing deep SQL expertise. Query builders offer a more abstract way of interacting with databases, allowing users to create complex queries without dealing directly with SQL command intricacies.[1]

Types of SQL Query Builders

Programmatic Query Builders

Programmatic query builders are libraries that developers use to generate SQL queries within their code. These tools provide a structured API that allows developers to build queries using method chaining and function calls rather than writing raw SQL strings. The structures and actions supported by the database layer remain recognizable, keeping developers relatively close to the data while providing programmatic convenience.[2]

Visual Query Builders

Visual query builders provide graphical user interfaces that enable users to construct SQL queries through drag-and-drop interactions, point-and-click operations, and visual selection of database elements. These tools are designed for both developers and non-technical users, featuring intuitive interfaces that display database schemas and allow users to select tables, columns, and define relationships visually.[3][4][1]

Categories of Query Builder Approaches

Research identifies two fundamental approaches to query builder design:[5]

- **Data-oriented query builders**: These incrementally refine the query output, focusing on the data transformation process
- **Syntax-oriented query builders**: These incrementally assemble the SQL syntax tree, focusing on building the SQL structure itself[5]

Popular SQL Query Builder Tools

The market offers numerous SQL query builder solutions, each with distinct strengths:

Enterprise-Grade Tools

- **Redgate SQL Prompt**: Provides contextual predictive text and code suggestions with over 90 built-in rules for query optimization[6]
- **dbForge Query Builder**: Features visual query diagrams, drag-and-drop functionality, and supports complex SQL queries with sub-queries[6]
- **Idera Rapid SQL**: Offers visual query building with contextual recommendations and supports multiple database management systems[6]

Cloud-Based and Modern Solutions

- **ThoughtSpot**: Features AI-powered search for SQL query generation with live query execution across multiple cloud data warehouses[7]
- **Active Query Builder**: Provides a no-code drag-and-drop interface with API capabilities for embedding into applications[7]
- **Metabase**: Offers visual query building that makes it easy for non-technical users to create queries through clicks and menu choices[8]

Interactive Query Interfaces

Interactive query interfaces go beyond traditional query builders by incorporating dynamic feedback mechanisms and user guidance systems. These interfaces often include:

Form-Based Interfaces

Interactive Query Form (IQF) systems dynamically generate query forms at runtime, allowing users to iteratively refine their queries through two main interaction types: Query Form Enrichment and Query Execution. These systems use metrics like F-measure to estimate query

form effectiveness and provide compressed result tables to avoid overwhelming users with large datasets.[9]

Natural Language Interfaces

Advanced systems like NaLIR (Natural Language Interface for Relational databases) accept complex English language sentences as query input and translate them into SQL queries. These systems include interactive communicators that explain query processing steps and offer multiple interpretations for user selection.[10][11]

Visual Selection and Brushing

Some interfaces support dynamic queries through widgets like range sliders and brushing techniques that enable selection through direct manipulation. Multi-dimensional brushing allows users to select data points visually, with the system automatically computing hypercubes that enclose selected points across all dimensions.[12]

Key Benefits and Use Cases

SQL query builders and interactive interfaces provide several advantages:

- **Accessibility**: Enable non-technical users to query databases without SQL knowledge[13][6]
- **Productivity**: Reduce development time through automated code generation and visual design[4][6]
- **Error Reduction**: Provide syntax checking, validation, and guided query construction[4][6]
- **Schema Awareness**: Display database structures and relationships to guide query construction[3][4]
- **Multi-Database Support**: Many tools support various database management systems with appropriate SQL dialect handling[7][6]

Integration with Modern Data Platforms

Modern query builders increasingly integrate with cloud data platforms and support multiple database types simultaneously. Tools like K2View's Query Builder allow querying multiple data source interfaces simultaneously, opening separate tabs for each interface. This capability is particularly valuable in modern data architectures where data spans multiple systems and platforms.[14][7]

The evolution of SQL query builders reflects the broader trend toward democratizing data access, making sophisticated database querying capabilities available to a wider range of users while maintaining the power and flexibility that technical users require.

⁂

I want to build a visual query builder in Databricks with Databricks apps

To build a **visual query builder** in Databricks using Databricks Apps, you have several options depending on your desired level of customization and user experience:

1. Use Databricks Built-In Visualizations and Dashboards

Databricks natively provides a visual experience for creating and editing queries—users can construct SQL queries using the SQL editor or notebooks, and then build visualizations and dashboards with a few clicks:

- Write a SQL query (manually or with autocomplete assistance).
- Click the "+" in the result panel to open the visualization editor, choose your chart type, and configure the display.
- Add the visualization to a dashboard and allow parameterized filtering for more interactivity.[21][22][23][24]
- This is ideal for standard analytics and BI needs, but does not provide **drag-and-drop** query composition or point-and-click query construction like some BI tools.

2. Build a Custom Visual Query Builder App with Databricks Apps

If you need a more interactive, low-code/no-code experience (e.g., drag-and-drop SQL query construction, similar to Microsoft Power BI or Metabase), you can create a **custom Databricks App**:

How It Works

- **Databricks Apps** let you deploy Python (Streamlit), or any web-based UI, directly within the Databricks workspace for your users.[25][26]
- Build your UI using open-source frameworks like Streamlit, React, or Flask, and integrate with Databricks SQL using its APIs.

Example Approach

1. **Develop a custom UI** for your query builder:
    - For a true visual query builder, use React/Streamlit to present tables, columns, join options, and filters to users.
    - Generate SQL queries on the fly as users select schemas/tables/fields.
2. **Integrate with Databricks**:
    - Use Databricks' REST API, ODBC/JDBC, or Python libraries to execute queries and fetch table/field metadata.
3. **Deploy the app** as a Databricks App:
    - Upload your code via the UI (Sidebar: New > App > Custom).[25]
    - Manage user permissions and sharing through Databricks workspace settings.
    - See examples and templates from Databricks' GitHub app-templates repo.[27]

Key Features You Can Build

- Drag-and-drop or point-and-click query construction.
- UI for selecting catalogs, schemas, tables, columns, filters, and joins.
- Real-time SQL preview/editing.
- Live query results returned and visualized within the same interface.[28]
- Save, edit, and share queries or dashboards.

3. Connect Existing Visual Query Builders via ODBC/JDBC

You can also use 3rd-party tools with visual query builders, connecting to Databricks as the backend:[29][30]

- Examples: Grafana, Active Query Builder (WinForms .NET UI), Tableau, or Metabase.

- Build queries visually in those tools and run them against Databricks SQL warehouses via standard connectors.

Resources to Get Started

- Databricks Apps developer docs[26][25]
- "Build and share a chat UI with Databricks Apps" (includes Streamlit example and deployment via CLI)[31][27]
- Example UI frameworks: Streamlit (Python-based rapid UI), React (JavaScript, for more advanced UIs)
- Use Databricks REST API or Python client for backend integration

Summary:
You can quickly enable visual query building with Databricks' built-in dashboarding and visualization tools. For a richer and more guided experience, leverage Databricks Apps to build your own drag-and-drop visual query builder using Streamlit, React, or other web frameworks, integrated with Databricks' APIs. Alternatively, connect Databricks SQL to external visual query builder tools for similar functionality.[31][27][26][28][25]



Do a user study on social and boards, is there a market for this type of tool.

There is strong evidence from user discussions, industry boards, and market analysis indicating a robust and growing market for visual query builders and low-code/no-code SQL tools.

Market Demand and User Sentiment

1. Explosive Market Growth

- The SQL query builders market is projected to grow from $1.2 billion in 2024 to $3.0 billion by 2033, with a CAGR of 12.6% from 2026 to 2033. Cloud-based and AI-powered

solutions are significant growth drivers, as enterprises strive for efficiency and real-time data access.[41][42]

2. Active User Interest Across Social & Developer Platforms

- Discussions on Stack Overflow, Reddit, and industry forums consistently reference both the pain points (complexity of hand-written SQL, need for rapid ad hoc queries) and positive reception for visual query builders. Many users seek free or open-source drag-and-drop alternatives due to licensing costs, posting frequently about tools like dbForge, SQLyog, and Active Query Builder.[43][44]
- User comments reveal that visual query builders help both technical users (speeding up routine queries) and non-technical users (democratizing access to data and reports). There are frequent requests and recommendations for tools with modern interfaces and solid support for database connectivity and cross-system integration.[45][44]

3. Adoption by Non-Technical Audiences

- The central value proposition of visual query builders is that they open up self-service analytics to business users, social media managers, marketers, and operational teams who lack SQL expertise. Special emphasis is placed on enabling users to “describe what they want” in plain English, which is increasingly delivered via AI integrations (see AI2SQL, SQLAI.ai).[46][47]

4. Positive ROI and Productivity Gains

- User stories and vendor marketing highlight dramatic time savings (e.g., "Save 10+ hours weekly," “Reduce query bottlenecks, errors by 95%”) and improved compliance and auditability through standardized query templates and results.[46]

Key Insights from Community Boards & User Studies

- **Reddit/Stack Overflow:** Extensive discussions exist about the need for user-friendly query builders, frustration with vendor lock-in/pricing, and a clear divide between tools built for engineers and those built for business analysts.[44][43]
- **DEV and ReScript Forums:** Developers often debate best practices (builder patterns vs. factories), indicating ongoing tool innovation for custom developer-centric query builders.[48][49]

- **LinkedIn and Market Reports:** Enterprises and SMEs are driving market expansion, with large tech firms (Microsoft, Oracle, Google, AWS, SAP, etc.) and innovative startups targeting the broader business audience with visual, AI-powered query solutions.[42][41]

Industry-Specific Demand

- Sectors including finance, retail, healthcare, e-commerce, and social media analytics require robust, visual querying for their massive, ever-growing datasets. The emphasis on compliance, real-time reporting, and self-service access is especially strong in data-driven verticals.[42]

Conclusion

There is a clear and accelerating market demand for visual query builders—particularly those that leverage AI, support multi-cloud data sources, and cater to non-technical users. Community sentiment and user studies suggest that successful solutions should emphasize ease of use, database connectivity, pricing flexibility, and strong audit/compliance features. This is a fast-growing segment with considerable opportunity for innovation and adoption.[50][45][42]



---

what specificlly would make an awesome product in this space that fits perfectly within databricks and can be built by databricks apps with cursor

To create an exceptional visual query builder as a Databricks App with Cursor that seamlessly fits the Databricks ecosystem, focus on blending data democratization with powerful analytics, leveraging the unique features of Databricks and user needs voiced in the market:

Key Features for an Outstanding Product

1. Intuitive, Drag-and-Drop Query Interface

- Allow users to visually explore catalogs, schemas, tables, and joins.

- Implement drag-and-drop to add tables, auto-suggest joins based on foreign keys, and show visual relationships.
- Enable filter, group, and sorting options—all without requiring SQL writing.

2. AI-Assisted Query Generation & Enhancement

- Integrate Cursor's AI to convert natural language questions or prompts into SQL queries with editable visual previews.
- Offer smart recommendations, auto-completion, data type hints, and auto-fix of likely SQL mistakes in real-time.
- Allow iterative refinement: "Show me sales by region," then "filter to Midwest," etc.

3. Full Support for Databricks Lakehouse Features

- Surface Lakehouse data lineage, catalog governance policies, Delta Lake tables, Unity Catalog security roles, and metadata directly in the UI.
- Enable querying across all Databricks environments (SQL Warehouse, Unity Catalog, and data in Files).

4. Collaborative, Shareable, and Secure

- Allow users to save, version, and share queries, visualizations, and dashboards securely within workspace projects.
- Enable collaborative editing and real-time comments leveraging built-in Databricks permissions.

5. Instant Visualization and Export

- Automatically visualize query results (bar, line, map, pivot, etc.) with one click.
- Support result export to Excel, PowerPoint, or Databricks dashboards.

6. Seamless Integration with Notebooks, Jobs, and Workflows

- Allow pushing generated queries into existing Databricks notebooks (Python, SQL, Scala) and jobs.

- Enable users to schedule queries and trigger actions (alerts, workflow starts) based on results—all managed through the UI.

7. Real-time Data Catalog, Governance & Compliance

- Expose Unity Catalog lineage, intended use tags, data owner information, and access policies directly in the builder UI, so users can make informed, compliant choices.

8. Built as a Native Databricks App

- Deployed via the Databricks Apps platform with tight workspace integration.
- Leverage Databricks APIs for authentication, metadata, permissions, and query execution.
- Use Streamlit (for rapid UI) or React (for advanced needs); integrate Cursor's AI via API for NLP + SQL assistance.
- All computation and sensitive data remain within Databricks' secure and governed boundaries.

What Sets This Apart

- **Democratizes Data Access:** Business users and data scientists can both self-serve, reducing bottlenecks on analytics teams.
- **Governance First:** Deep Lakehouse integration ensures all data access is audit-logged, secure, and compliant.
- **AI Native:** Combining Cursor's NLP/AI capabilities makes it easier, faster, and less error-prone than legacy query builders.
- **Fully Extensible:** Can connect to partner or customer APIs to include context-aware templates, business metric catalogs, or reusable analytics logic.
- **No Vendor Lock-in:** All logic and data stay in Databricks, ensuring future-proofing and enterprise trust.

Example User Flow

1. **User launches the Visual Query Builder app from the Databricks sidebar.**
2. **Navigates to Unity Catalog view; drags tables into workspace.**
3. **Cursor's AI suggests joins, computes sample queries, and visualizes expected results.**

4. **User tweaks filters and summaries visually, optionally edits generated SQL if needed.**
5. **With a click, results are plotted, shared, and saved; can schedule as a job or export as a dashboard.**

Conclusion

A visual query builder built as a Databricks App with Cursor would empower users of all skill levels to explore, analyze, and share insights from the Lakehouse, fully leveraging Databricks native governance, security, and performance—all accelerated by modern AI, and with zero compromise on compliance or security. This perfectly addresses current market gaps and aligns with user feedback for what's missing in both Databricks and the data tooling ecosystem.

I think the most important peices for a MVP are catalog integration, data profiling/preview pane, canvas with click drag and drop for query elements, and live preview if we have time, performance analysis what do you think?

You have identified the most critical components for a Minimum Viable Product (MVP) for a visual query builder on Databricks:

1. Catalog Integration

- **Absolutely essential.** Deep integration with Unity Catalog ensures that users can browse, search, and select tables, views, and columns directly from your app. This not only leverages Databricks' strong governance and access controls but also provides users with metadata (schemas, descriptions, permissions) at their fingertips.
- **Value:** Enables compliance, security, and discoverability—core reasons organizations adopt Databricks.

2. Data Profiling / Preview Pane

- **Highly important.** Users need to quickly see sample data, data types, row counts, distributions, and basic stats before constructing queries.

- **Value:** Reduces errors, improves query accuracy, and builds trust by showing users what's actually in the data.

3. Canvas with Click, Drag, and Drop for Query Elements

- **Core differentiator.** The drag-and-drop visual interface is what truly democratizes query building and sets your app apart from the built-in SQL editor.
- **Value:** Lowers the technical barrier, increases adoption among analysts/business users, and speeds up query construction even for power users.

4. Live Preview (if achievable)

- **Highly desirable, even if "lite."** Running queries as users build them—showing result counts or sample records—makes the app feel dynamic and responsive.
- **Value:** Instant feedback reduces trial-and-error, helps users catch mistakes, and makes building complex queries much less intimidating.
- **Warning:** This can be resource-intensive on large datasets. Limit preview to first N rows, sample-based metrics, or estimated result sizes for the MVP.

5. Performance Analysis (if possible in Phase 1)

- **Nice to have, but lower priority for MVP.** If you can expose basic query cost estimates (scan size, rows processed, warnings for full table scans or cross joins), it's a strong value-add.
- **Value:** Helps users create efficient queries and reduces backend costs, but not strictly necessary if resourcing is tight for MVP.

Recommendation

Focus your MVP on:

- **Seamless Catalog integration**
- **Often-overlooked data profiling/preview**
- **Intuitive drag-and-drop canvas**

If you have the bandwidth, **layer in live preview** ("sample rows" or "estimated row count")—this will really impress users and cut down on back-and-forth errors.

Performance analysis can follow as an enhancement or “pro tip” feature after launch. For now, ensure you design with extensibility so it’s easy to add later (e.g., by querying Databricks’ query history, or by analyzing explain plans in future iterations).

This approach targets the true “painkillers” that users highlight in both enterprise and community feedback, delivering immediate value while providing a clear, adaptable roadmap for future sprints.

---

give me a prompt to give cursor to build our mvp given this context, be as detailed and specific as possible. ask any follow up questions about the build. I want a modern look that is aesthetically pleasing and super user friendly

Here’s a detailed prompt you can give to Cursor to kick off your MVP build. I’ll also include clarifying questions you may want Cursor to ask/answer before starting implementation.

Prompt for Cursor:

We are building a Minimum Viable Product (MVP) “Visual Query Builder” Databricks App to run natively in the Databricks workspace sidebar. The goals are: seamless catalog integration, rapid data discovery via preview/profiling, and an intuitive drag-and-drop canvas for building SQL queries with live query previews.

Requirements and Guidance:

1. UI/UX

- The app should have a modern, visually appealing, and highly usable interface suitable for Databricks enterprise customers (inspired by Figma, Hex, dbt Cloud, or Retool).
- The sidebar shows the Unity Catalog hierarchy (catalogs, schemas, tables/views).
- The main “canvas” area lets users drag and drop tables/views, join them visually (auto-suggestion of FK joins), and add columns, filters, and sorts interactively.

- Modern theming, responsive layout, accessible controls, and elegant use of whitespace and iconography.
- Include explicit support for dark mode.

2. Catalog Integration

- Connect to Unity Catalog using Databricks REST API (or best practice).
- Support browsing/searching datasets, clicking to expand table metadata (columns, types, comments, row counts, partition/sort keys, last updated info).
- Allow table previews—display the first N rows/sample data with “Profile” facts (row count, distincts, null counts, min/max, top freq values for each column).

3. Data Profiling/Preview

- Data preview pane appears whenever a table/view is clicked or added to canvas.
- Show summary stats alongside column list (as a small info panel or modal).
- Limit per-request volume to keep the app fast and light (e.g., 20–100 rows or 1MB max).

4. Canvas & Query Builder

- Drag tables/views onto canvas and rearrange them visually.
- Draw connections by clicking and dragging from primary to foreign keys—auto-suggest plausible joins using catalog metadata.
- Users can mouseover or click to select columns, set filters (with UI for conditions), add group by/aggregations, and sort order.
- Dedicated “SQL Preview” pane with auto-generated, auto-updating SQL corresponding to the visual model.
- Optionally, let advanced users tweak/edit the SQL directly (bonus).

5. Live Query Preview (if time allows)

- When the query is valid, allow users to run a sample (with row limit) to return live data into an output pane.

- Handle and display errors gracefully, including SQL validation feedback and run cost estimates if feasible.

6. Workspace Integration

- All actions should respect Databricks workspace permissions and Unity Catalog governance.
- Provide option to export/save a query as a Databricks saved query, notebook cell, or dashboard.

7. Performance & Scalability

- Make preview queries and metadata sampling efficient—fail gracefully on large or partitioned tables.
- (If possible: display warning if full table scan or costly operation is likely.)

Design/Technology Notes:

- Prefer building in Python (Streamlit for Databricks Apps) or React/TypeScript (if supported).
- Should follow Databricks design system and feel like a native extension.
- Responsive and accessible; works on laptops and large displays.
- Cursor's code should emphasize clean, reusable components, and clear separation of catalog, canvas, and preview logic.

Follow-up/clarifying questions for Cursor:

1. What specific Databricks workspace and Unity Catalog access do you need for development and testing?
2. Do we need support for "viewing" Delta Live Tables, views, or materialized views, or just base tables in the MVP?
3. Should scheduling or saving queries to dashboards be in-scope for the MVP, or just export/copy options?
4. Will user authentication and roles be passed from the Databricks workspace context automatically, or is custom handling needed?

5. Are there brand guidelines, design tokens, fonts, or color palettes (besides Databricks defaults) we want enforced?
6. Should we design for expansion to supporting “AI chat to SQL” later, or stick to visual-only in v1?

Feel free to modify or emphasize any section. This gives Cursor both clear requirements and the context to ask smart follow-up questions—setting you up for a design and delivery process that delivers real value, fast.

---

research how people are building apps on databricks and improve the prompt

Based on best practices and the latest approaches to building Databricks Apps—gathered from Databricks documentation, developer blogs, community discussions, and platform updates—here’s a significantly improved and more actionable prompt for Cursor. This version incorporates not just requirements, but also modern engineering, security, and UX recommendations, and explicitly leverages Databricks-native strengths.

Prompt for Cursor:

We want to design and develop a “Visual Query Builder” MVP as a native Databricks App, purpose-built for Databricks’ workspace, governance, and performance features. The goal is to deliver a modern, delightful drag-and-drop query experience backed by seamless Unity Catalog integration, live data preview, and intuitive data profiling, all leveraging Databricks’ managed infrastructure and security.

Key MVP Features & Guidance

1. Tech Stack & App Architecture

- Use the preferred Databricks App frameworks (Streamlit for simplicity and speed, React for more advanced, customizable UX).
- Define app metadata and runtime in `app.yaml` (command, dependencies, environment).
- Ensure code modularity and clear separation between UI logic, data access, and integration points.

- Target serverless, fully managed operation in Databricks—no custom infra or manual compute provisioning.

2. UI/UX & Modern Aesthetics

- Follow Databricks' UI/UX guidelines for consistency (databricks.design system, responsive layouts, use of whitespace, and accessibility).
- Sidebar: Hierarchical tree for Unity Catalog (catalog/schema/table, searchable).
- Main panel:
    - Visually styled "canvas" workspace for drag-and-drop.
    - Tables can be dragged in, joined visually, and subsets/columns/filters applied.
    - Modern components: animated interactions, tooltips, dark mode support, chart-ready styling.
- Live, presentable result/data preview panel with sample rows/stats and clear error messaging.
- Minimize "reload" latency for data and sample previews (async requests, skeleton loaders, optimistic UI if needed).

3. Catalog & Data Security Integration

- All data access through approved Databricks APIs using the app's workspace context—respect user permissions, SSO, Unity Catalog governance.
- Expose metadata, lineage, and security tags as part of the catalog browser.
- Don't cache, log, or transmit user data outside Databricks.

4. Data Profiling & Preview

- When any table or join is selected, provide in-place profiling:
    - Basic stats (row count, unique/distinct/top values, null counts, data types, minima/maxima)
    - Show up to 100 sample rows, error gracefully for large tables.
- Efficient: Use sampling/small result sets to minimize warehouse load.

5. Drag-and-Drop Canvas/Query Builder

- Intuitive drag-and-drop UX: visually place tables, connect joins (suggest joins when possible from UC metadata).
- Allow column selection, filters (contextual UI for condition building), sorts, group-bys.
- Live auto-generation of SQL in real time with clear “edit” option for power users.
- Canvas should be keyboard accessible and support undo/redo if feasible.

6. Live Query Preview

- Optional, if time: provide a button to run the constructed query with a hard-coded limit and show results instantly.
- Clearly report run success/errors/cost/warnings.

7. Workspace & Native Databricks Integration

- App should be deployed as a first-class Databricks App (sidebar launch, workspace permissions, etc).
- Respect all workspace config, user context, and Unity Catalog controls.
- Allow users to save/export queries to Databricks Notebooks, Dashboards, or copy SQL to clipboard.

8. DevOps and Testing

- All code tracked in Git; automate deployment (use Databricks Asset Bundles/DABs for CI/CD).
- Include and clearly document unit tests for business logic & catalog integration.
- App logs only to stdout/stderr, no persistent files.

9. Caching and Performance

- Use in-memory caching (Python LRU, or React Query cache) for metadata and profile samples—scope per-session.
- Optimize for cold-start and runtime performance: minimize heavy dependency loading and prefetch if needed.

10. Security and Error Handling

- All user input validated and sanitized.
- Graceful global error boundaries; never expose raw stack traces to users.
- All compute runs must terminate quickly (<1 min) or show graceful timeout dialog.

Additional Above-and-Beyond (Optional if Feasible)

- Integration with Workspace “recent objects” or “favorites.”
- Show usage stats (last query run, most queried tables) to improve usability.
- Table/view/column descriptions inline for business context.
- Ready for “AI assistant”/search to SQL (future expansion, not MVP).

QUESTIONS FOR YOU (CURSOR):

1. Should we target Streamlit (rapid, accessible, less design control) or React (polish, animation, more control) for the main MVP build? Can you share experience with both within Databricks Apps?
2. What out-of-the-box Databricks UI components or libraries are available and recommended for modern, branded design?
3. What’s the recommended API for Unity Catalog and sample preview access—any latest/best practices for robust catalog browsing and fast previewing?
4. Should catalog metadata, lineage, or data samples be cached per user/session by default for speed?
5. What Unit/Integration test patterns do you recommend for Databricks Apps (esp. for API integration and UI state)?
6. Is there native clipboard/export support in the Apps SDK for exporting SQL, or should this be handled in the app directly?
7. What are your recommended methods for handling permissions, error messages, and large result set warnings gracefully?

Instructions:
Please review this spec, let me know if anything is unclear, and answer the above questions

before build-out. Feel free to challenge any requirements and suggest optimizations or alternatives based on recent successful Databricks App deployments.

This prompt directly follows leading practices and addresses both the user and developer experience, making for a more impactful and maintainable MVP.Here's an improved, detailed prompt for Cursor, based on how modern Databricks Apps are being developed (see current best practices, platform guidance, CI/CD, security, and UI/UX experience):

Prompt for Cursor:

We need to build a Minimum Viable Product (MVP) Databricks App:
A modern, highly usable Visual Query Builder that runs natively in the Databricks workspace.
It should make writing, running, and iterating on SQL visually easy for all users via drag-and-drop, profiling, and instant preview.

Requirements—incorporate these Databricks-native engineering best practices:

1. App Structure & Tech

- App should run as a Databricks native App (deployed via sidebar, supporting serverless, secure execution).
- Use Streamlit for rapid prototyping, or React for advanced, pixel-perfect UI—let us know which fits our visual/canvas+datagrid needs best.
- All app logic, config, and dependencies managed via `app.yaml` and pinned `requirements.txt` (or package.json).
- Workspace authentication, permissions, and Unity Catalog enforcement must be automatic.

2. Modern UI/UX

- Adhere to Databricks design system (spacing, dark/light modes, icons, etc.); responsive UI for all displays.
- Sidebar: browsing/search of full Unity Catalog (catalog, schema, table, view), including search/filter, and icon-based navigation.

- Main area/canvas: intuitive drag-and-drop zone for query composition—tables can be dropped, columns picked, joins visually created (auto-suggestions on PK/FK keys when available).
- Highlight smooth transitions/animations and hover/tooltips (look to Figma, Hex, Mode for design inspiration).
- Must be aesthetically beautiful and easy for new users.

3. Catalog & Data Profiling

- All table/view access by Unity Catalog API, safe under workspace user context (no extra creds).
- When user clicks/positions a table, fetch and display:
    - Data profiling: row/column count, distinct values, nulls, min/max, type distribution per column.
    - Data preview (limit to N rows, ~50-100 for speed/security).
- Use “skeleton” loading or animated placeholders for smooth, optimistic UI.

4. Drag-and-Drop Query Logic

- Canvas should allow:
    - Dragging in multiple tables/views, auto-joining (suggest joins via metadata).
    - Selecting columns visually (checkbox or slider UI).
    - Adding and editing filters and group-bys using easy, click-based UI.
    - Live “Generated SQL” panel, always up to date with the visual state (SQL read-only for now, editing is a nice-to-have).
- Support keyboard accessibility and simple drag handles.

5. Live Preview (MVP/phase-1)

- Run constructed queries and show result preview (limit runtime and rows for warehouse efficiency).
- Feedback and errors must be shown through clear, friendly dialogs, never raw server errors.

6. Workspace Integration

- Make saving/exporting queries to Databricks dashboards, notebooks, or clipboard as easy as possible.
- All actions respect built-in workspace permissions and access control.
- No external data export—must stay in Databricks for security.
- Session-level caches for metadata/query previews to limit API calls.

7. Deployment, DevOps & Testing

- All code in Git; automate CI/CD deployments (Databricks Asset Bundles if possible, Github Actions, etc).
- Pin dependencies, avoid latest versions.
- All logs to stdout/stderr only.
- Global try/catch for errors; return safe, friendly user-facing messages.

Questions for Cursor Before Build:

1. Streamlit vs React: which do you recommend for a visually sophisticated, drag-canvas query builder with data grid/preview?
2. What Databricks UI components/libraries best match the modern, branded look we're after?
3. Is session-based caching of metadata and profiling data recommended in Apps for speed?
4. For catalog/profile APIs, what's the fastest/best practice for preview/sample data?
5. How are large query result warnings/limits best surfaced in-app?
6. Do you recommend any built-in testing or CI/CD integration patterns for Databricks Apps?
7. Is there a low-lift way to add keyboard accessibility to the main canvas/preview areas?

Please review, suggest improvements, and confirm your understanding before you start building.

Let us know if any requirement would slow down MVP delivery or if you see optimizations for Databricks-native Apps.

This refined prompt will help elicit a more robust, modern, and “real” product from Cursor, reflecting actual user/developer pain points, Databricks integration, and UI best practices.

⁂

---

can you research those questions and answer them if you can, ask me if there is a choice that is not clear and straight forward

Here are research-backed answers to your key architectural and design choices for a Databricks App-based visual query builder MVP—drawn from recent Databricks documentation, developer experience, and product blogs. For any topic with multiple valid options, I’ll highlight and ask for your input.

1. Streamlit vs. React for Databricks Apps (Especially Drag & Drop Canvas)

Streamlit:

- **Pros:** Rapid prototyping, all-Python workflow, easy data visualization, simple app structure, robust support in Databricks Apps.
- **Cons:** Layout and styling are relatively rigid, and deep visual customizations (like advanced drag-and-drop canvases or highly bespoke controls) are difficult without custom HTML/JS/CSS. Every interaction reruns the backend script, sometimes slowing responsiveness in complex apps.[81][82]
- **Best for:** MVPs, dashboards, analytic tools, forms, and wizard-like flows.

React:

- **Pros:** Fully customizable modern UI, pixel-perfect layout, animations, rich interactivity, supports sophisticated UI elements (true drag-and-drop canvas, live data grids, rich widgets).[83][84]
- **Cons:** Requires a backend (FastAPI, Flask, etc.) for computation/data access; more engineering overhead but much more flexibility.[83]
- **Best for:** Apps needing a heavy, modern, highly interactive UX.

Recommendation:
Since your vision requires a “super user-friendly” and modern canvas (drag tables, join visually, live profiling, etc.), **React is likely the best fit**. Streamlit is great for MVPs but can become limiting for complex UI/drag-and-drop. If you want speed to first prototype, start with Streamlit and migrate to React, or go React-first for design flexibility.[84][81][83]

> **Do you prefer to prioritize rapid MVP speed (Streamlit) or full visual sophistication (React)?**

2. Databricks Apps UI Component Libraries

- JavaScript/React apps in Databricks often use popular open UI libraries—e.g., **shadcn/ui** (well-styled), Material UI, Ant Design—for accessible, beautiful components.[84]
- For Python apps, Streamlit provides out-of-the-box widgets, but extensibility is limited compared to JS.
- Use Databricks design tokens and styles for workspace-native look and feel (consult Databricks design system docs, if available).

3. Session Caching Best Practices

- All sensitive data and catalog info should remain within Databricks.
- Use in-memory caching per user session for metadata and catalog profile samples to minimize API calls and accelerate the UI.[85]
- For query data: Databricks provides fast disk (Delta) caching for tables, useful for accelerating repetitive sample/preview reads.[86][87]
- No persistent app-level caching outside Databricks; all cache/data strictly scoped to session and user security.

4. Fastest Methods for Catalog/Profiling/Preview APIs

- Use Unity Catalog APIs for structured exploration (catalog/schema/table/view hierarchies).
- For preview and profiling:
  - Use `SELECT * FROM table LIMIT N` (N = 50–100) for samples.

    - Aggregate queries for profiling (min/max/nulls/count/distinct) per column; batch these for efficiency.
    - Where available, leverage Delta table/Unity Catalog summary stats and data lineage features.[88]
- Optimize API calls: parallelize async where possible, cache summaries per session.

5. Handling Large Query Result Warnings

- Use Databricks **Query Watchdog** to automatically prevent runaway queries and large result sets.[89]
    - Configure output limits (e.g., n rows, ratio of output to input) via cluster settings.
    - Surface “result too large” warnings clearly in the app if the watchdog or backend blocks/aborts the query.
- For user experience:
    - Always cap preview and downloads (e.g., 100 rows).
    - Display informative, actionable error messages (not stack traces!)—e.g., “Query exceeded allowed result size, please add filters or reduce data returned.”

6. CI/CD and Testing Patterns for Databricks Apps

- Use **Databricks Asset Bundles (DABs)** for packaging app code, assets, config—enables dev-test-prod workflows and easy deployment to multiple workspaces/environments.[90][91]
- Use Nutter or pytest for unit/integration testing in Databricks environments; test both business logic and data API interaction.[90]
- Automate builds with GitHub Actions or similar, deploying bundles via CLI.
- Best practice: maintain isolated workspaces for staging, production, etc.

7. Keyboard Accessibility in Databricks Apps

- For React/JS apps, prioritize accessible component libraries (all buttons, menus, dialogs, canvas zones navigable via **Tab**, **Enter**, **Space**, etc.).[92]
- Ensure proper focus management, clear visual focus indicators, and ARIA attributes.
- Test with screen readers and keyboard navigation during QA.

- Streamlit: accessibility is improving but more limited, especially for custom or non-standard widgets.

Summary Table for Architectural Choices

Topic	Recommendation	Reasoning
Core UI Framework	React	Best for drag-and-drop, high UX customization[83][84]
Component Libraries	shadcn/ui, MaterialUI, Databricks design tokens	Modern, user-friendly, accessible[84]
Data Caching	Session memory, Databricks Delta cache	Fast, secure, minimal persistence outside user session[86][87]
Profiling/API	Unity Catalog APIs, SQL LIMIT, batch stats	Performance, metadata accuracy, Leverage Databricks native[88]
Result Warnings	Query Watchdog, front-end controls	Protect resources and improve user experience[89]
CI/CD	DABs, Nutter, pytest, GitHub Actions	Robust, scalable deployments and testing[91][90]
Accessibility	Prioritize accessible libraries, test thoroughly	Crucial for enterprise adoption[92]

If you have a strong preference for rapid time-to-first-prototype, Streamlit is defensible (can migrate later). For a truly best-in-class drag-and-drop UX, React is the clear winner and supported natively in Databricks Apps. Otherwise—all other choices are clear best practices.

Let me know which framework you want to target, and I'll refine the build plan with these answers!

⁂

---

is anyone else working on something like this

There is **currently no official first-party drag-and-drop visual query builder app purpose-built for Databricks Apps** (i.e., natively in the workspace sidebar, leveraging Unity Catalog, session context, and Databricks UI/UX conventions).

However, there are related efforts and integrations in the broader Databricks ecosystem and community:

1. Native Databricks Solutions

- **Databricks SQL Editor**: Supports catalog browsing, autocomplete, visual query construction aids, and easy addition of tables/columns, but not drag-and-drop/canvas-style querying. It is not a full visual query builder and doesn't support advanced visual join or diagramming logic like you're proposing.[101][102][103]
- **Databricks Apps Framework**: Common use cases include dashboards, analytics explorers, and data-quality tools, but as of now, there's no marketplace or canonical example for a visual SQL builder app in this framework.[104]

2. Third-Party and Community Projects

- **Grafana's Databricks SQL Query Builder**: Grafana's Databricks connector features a schema-aware builder for query generation (filters, groups, preview), but this is within Grafana, not as a Databricks App or inside the Databricks UI.[105][106]
- **Active Query Builder**: Enables use of a visual drag-and-drop query interface for Databricks data, but via standard APIs (ODBC/JDBC) in .NET/WinForms—external to Databricks' web interface.[107]
- **DBeaver, DataGrip, Appsmith, Alteryx, Redash, etc.**: All support Databricks connectivity and have some visual query design features (see DBeaver Visual Query Builder), but they run outside Databricks and aren't workspace-native.[108][109][110]
- **Appsmith**: Supports Databricks data sources and has drag-and-drop UI for business apps/dashboards, but this is not a plug-and-play Databricks-native App.[109]

3. Open Source & Community Discussions

- **GitHub projects** such as "databricks-grafana" provide simple query builders as add-ons/connectors but not implemented as full Databricks Apps with sidebar, Unity Catalog integration, and deep workspace context support.[106]

- **Reddit and developer forums** regularly discuss the need for such tools; for example, TypeScript projects seek a Databricks-compatible query builder, but no widely shared solution exists in the App framework.[111]

4. Summary Table: What Exists and What Doesn't

Solution	Visual/CANVAS UI	Databricks-native Sidebar App	Unity Catalog Integration	Join Suggestion	Marketed as Databricks App
Databricks SQL Editor	✖	✔	✔	✖	✔
Grafana Builder (Plugin)	✔	✖	✔	✔	✖
DBeaver/External IDEs	✔	✖	Via SQL only	✔	✖
Appsmith	✔	✖	✔	✖	✖
Custom (GitHub, forums)	(Some)	✖	Sometimes experimental	Sometimes	✖
A true Databricks Apps MVP	**Your Idea**	**Yes**	**Yes**	**Yes**	**Planned**

Conclusion

Nobody has shipped a Databricks App (in-sidebar, Unity Catalog-aware, drag-and-drop/canvas visual query builder as you described) as an open-source or commercial project yet. Existing solutions are either inside other products (Grafana/Appsmith), only partially visual, or aren't deployed as Databricks Apps.

This is a powerful whitespace opportunity. You will be first with:

- Deep Unity Catalog integration
- Databricks-native user/session/permission handling
- Drag-and-drop/canvas query builder—enterprise-grade, not just code-completion
- Workspace-native deployment and Databricks UI/UX consistency

If you proceed now, you will be leading this category inside the Databricks Apps ecosystem.

⁂

---

in databricks apps, are there self referencing parameters or global variables to idenitfy the workspace token warehouse or anything like that to leverage the deplolyment

In Databricks Apps, you can leverage *environment variables* and *resource references* to identify and utilize workspace-level context (such as workspace hostname, user/session, and declared resource IDs) at deployment and runtime:

Environment Variables Automatically Set by Databricks

- **Databricks automatically sets crucial environment variables** within the app runtime. These include, among others:
  - `DATABRICKS_HOST` (your workspace URL)
  - `DATABRICKS_TOKEN` (the API token used by the app's authentication context)[121][122][123][124]
- These variables are accessible in your app code and can be used to authenticate and interact with APIs, and are set securely without hard-coding secrets.

App Resource References

- You can declare workspace resources (such as **SQL Warehouses, Jobs, Volumes, Secrets**) in your app's configuration (`databricks.yml`), specifying them as dependencies needed at runtime.[125]
- **At deployment time**, workspace admins connect these references to actual resource instances via the UI—this allows your app to be environment-agnostic and never hard-code IDs for production, staging, or dev.[125]

- **At runtime**, your app can access the IDs and connection info for these resources using variables automatically passed by Databricks.

Substitutions & Context Variables (DABs)

- When packaging with Databricks Asset Bundles, you can use substitution variables such as:
  - `${workspace.host}` – URL/hostname of the active workspace
  - `${workspace.current_user.userName}` – current user running/deploying/package
  - `${resources.sql_warehouses.<name>.id}` – unique resource (e.g., warehouse) ID
  - `${bundle.name}`, `${bundle.target}` – for environment-specific deployment differentiation[126][127]
- These can be used in app config, deployment scripts, or programmatically.

Dynamic Value References (Jobs/Workflows)

- In Databricks Workflows (not directly in apps, but in jobs/pipelines) you can use `{{job.id}}`, `{{tasks.<task_name>.output.*}}`, and similar to inject dynamically-resolved values into parameters or downstream tasks, but not inside raw code—these must be declared in the orchestrator/config level.[128][129]

Practical Application (in Apps)

- **Don’t hardcode resource IDs or tokens.** Get the current workspace, warehouse, user, etc., via config or env variables.[130][124]
- **Access the SQL warehouse/"endpoint" as a declared app resource:** At deployment, the app config will be updated to reference the actual warehouse for the current workspace/session.[125]
- **Access current workspace, user, and other context** via environment variables or substitutions.
- **Use secret injection** for sensitive values (tokens, passwords)—these can be referenced securely in your app config.[130][125]

Example: Using Environment Variables in App Code (Python)

```
import os

workspace_host = os.environ["DATABRICKS_HOST"]
api_token = os.environ["DATABRICKS_TOKEN"]
warehouse_id = os.environ.get("DATABRICKS_WAREHOUSE_ID")  # if declared as a resource in
config
```

Conclusion

Yes: In Databricks Apps, you can reliably identify and leverage self-referencing parameters and global/environment/context variables for:

- Workspace hostname & info
- Auth token (secure, never hard-coded)
- Resource IDs (warehouse, job, secret, etc.)
- User, deployment, and environment context

This makes your app portable, secure, and seamless across workspaces/environments—no hard-coding or manual edit required at deployment time. If you have a specific variable or scenario in mind, let me know and I can advise how to access it![122][123][124][121][126][130][125]



---

this is what i got back from cursor. How do i configure my local machine to be able to launch this? help me setup a venv and put the env variables in

Here's a step-by-step guide to help you configure your local machine to develop and run your Databricks App with a Python virtual environment (**venv**) and the necessary Databricks environment variables:

1. Set Up Python Virtual Environment (venv)

a. Open your terminal or command prompt.

b. Navigate to your project directory (or create one):

```
mkdir my-databricks-app
cd my-databricks-app
```

c. Create a virtual environment:

```
python3 -m venv venv
```

(for Windows, use python -m venv venv)

d. Activate the virtual environment:

- **macOS/Linux:**

  ```
  source venv/bin/activate
  ```

- **Windows:**

  ```
  .\venv\Scripts\activate
  ```

Your terminal prompt should now start with `(venv)`.

2. Install Your Application's Dependencies

If there's a `requirements.txt` in the app code, install dependencies:

```
pip install -r requirements.txt
```

(Add any extras you want with pip install <package>.)

3. Set Up Databricks Environment Variables

You'll need:

- Your Databricks **workspace host URL** (e.g., `https://dbc-xxxx.cloud.databricks.com`)
- A Databricks **personal access token** (generate from User Settings > Developer)

Option A: Set env variables temporarily in your shell:

```
export DATABRICKS_HOST="https://dbc-xxxx.cloud.databricks.com"
export DATABRICKS_TOKEN="dapiXXXXXXXXXXXXXXXXXXXXXXXX"
```

(On Windows Command Prompt, use `set` instead of `export`. On PowerShell, use `$env:DATABRICKS_HOST = "..."`.)

Option B: Store in a `.env` File (Recommended for Local Development)

1. Create a file named `.env` in your project root:

   ```
   DATABRICKS_HOST=https://dbc-xxxx.cloud.databricks.com
   DATABRICKS_TOKEN=dapiXXXXXXXXXXXXXXXXXXXXXXXX
   ```

2. Install a package like `python-dotenv` if your app loads environment variables from `.env`:

   ```
   pip install python-dotenv
   ```

3. At the top of your main Python script, add:

   ```
   from dotenv import load_dotenv
   load_dotenv()  # This will read .env file and set variables
   ```

4. (Optional) Set Data Warehouse or Resource IDs

If your Cursor code expects other environment variables (like `DATABRICKS_WAREHOUSE_ID`):

- Add them to your `.env` or export them like above.

Example line in `.env`:

```
DATABRICKS_WAREHOUSE_ID=1234-567890-warehouse
```

5. Run Your App

Now, with your venv activated and env variables set or loaded, run your Python app:

```
python main.py
```

(or whatever the entrypoint script is, e.g., `app.py` or via `streamlit run app.py`.)

6. Quick Check

To confirm your env variables are loaded, you can print in a test file:

```
import os
print("Workspace host:", os.environ.get("DATABRICKS_HOST"))
print("Token present:", bool(os.environ.get("DATABRICKS_TOKEN")))
```

Let me know your OS and if you want more automation (like a script that sets everything up). I can adapt the steps for your specific project or IDE (VS Code, PyCharm, etc.) if needed!

---

this is the directory i have it built in locally

'/Users/franco.patano/cursor projects/visualquerybuilder'
can i use this same one for the python environment?

Yes, you can (and should) use your existing project directory (`/Users/franco.patano/cursor projects/visualquerybuilder`) to set up your Python virtual environment and manage your Databricks App's dependencies and code. This keeps everything organized and self-contained.

Here's exactly what you should do in your case:

1. Navigate to Your Project Directory

```
cd '/Users/franco.patano/cursor projects/visualquerybuilder'
```

(use quotes if your path has spaces, as above)

2. Create the Python Virtual Environment Inside This Folder

```
python3 -m venv venv
```

- This creates a `venv` folder in your project directory containing your virtual Python environment.

3. Activate the Virtual Environment

```
source venv/bin/activate
```

You'll see (venv) in your command prompt. This means all installs and Python runs are isolated to this project directory.

4. Install Project Dependencies

If you have a `requirements.txt` (or similar file):

```
pip install -r requirements.txt
```

Or add dependencies as needed.

5. Set Up Environment Variables

- Follow the earlier instructions to either export env variables in your shell or add a `.env` file in this folder (recommended!).
- Example `.env` (in `/Users/franco.patano/cursor projects/visualquerybuilder/.env`):

  ```
  DATABRICKS_HOST=https://dbc-xxxx.cloud.databricks.com
  DATABRICKS_TOKEN=dapiXXXXXXXXXXXXXXXXXXX
  ```

6. Run Your Application from This Directory

Your app code and your virtual environment should both live in this folder. So when running your app, make sure your terminal is inside this directory and that your venv is activated.

```
python main.py
# or your entry point
```

Summary:
It is best practice to keep your project code and its venv together. This ensures all your installs, dependencies, and settings are isolated to this Databricks App project. Just be sure to add `venv/` to your `.gitignore` if you are tracking the project with git.

If you want, I can provide a fully scripted setup for your environment. Let me know if you're using an editor (e.g., VS Code) and if you want additional tips for that!

---

got this error

(venv) (base) franco.patano@NL9VV2GC9H visualquerybuilder % pip install -r requirements.txt

ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt'
WARNING: You are using pip version 21.2.4; however, version 25.2 is available.
You should consider upgrading via the '/Users/franco.patano/cursor projects/visualquerybuilder/venv/bin/python3 -m pip install --upgrade pip' command.
(venv) (base) franco.patano@NL9VV2GC9H visualquerybuilder %

The error message:

```
ERROR: Could not open requirements file: [Errno 2] No such file or directory:
'requirements.txt'
```

means that there is **no `requirements.txt` file** in your project directory (`/Users/franco.patano/cursor projects/visualquerybuilder`). This file is used to list all Python dependencies for your project, so that `pip install -r requirements.txt` knows what to install.

What To Do Next

1. Check if You Were Given a `requirements.txt`

- Double-check your Cursor output or any code you downloaded for this project.
- If Cursor/code you cloned/built did not generate `requirements.txt`, check if there are any instructions in a README or other onboarding file about required dependencies.

2. Create Your Own `requirements.txt` (if missing)

If you know which Python libraries your project needs (e.g., `streamlit`, `requests`, `python-dotenv`, `databricks-sdk`, etc.), you can create the file yourself.
Example:

```
streamlit
requests
python-dotenv
```

```
databricks-sdk
```

- Save this as `requirements.txt` in your project folder (`/Users/franco.patano/cursor projects/visualquerybuilder/requirements.txt`).
- Then rerun:

  ```
  pip install -r requirements.txt
  ```

If you're not sure what dependencies you need, check the top `import` statements in your project's main Python files, or share a list here and I can help compose a starter `requirements.txt`.

3. Upgrade pip (Recommended, Optional)

Your pip is quite out of date. To upgrade within your venv, run:

```
python -m pip install --upgrade pip
```

or

```
/Users/franco.patano/cursor\ projects/visualquerybuilder/venv/bin/python3 -m pip install --upgrade pip
```

Upgrading pip can help you avoid compatibility issues with newer libraries.

Let me know if you want help figuring out which dependencies to list in `requirements.txt` or have questions about your app's main libraries!

---

this was the response from cursor, i dont think we are doing this right

Perfect! I've successfully built a complete Visual SQL Query Builder as a Databricks App. Here's a comprehensive summary of what has been implemented:

🎉 Complete Implementation Summary

✅ Core Components Delivered

🗂️ Catalog Explorer Panel

Unity Catalog integration with hierarchical browsing

Search functionality across catalogs, schemas, tables, and columns

Drag-and-drop support for tables and columns

Real-time metadata loading with loading states

📊 Data Profiling & Preview Pane

Comprehensive data profiling with statistics (row counts, null counts, unique values)

Interactive sample data display

Distribution histograms and charts using Recharts

Column-level profiling with data type information

🎨 Visual Canvas

Drag-and-drop interface powered by React Flow

Table nodes with column visibility controls

Visual join creation by connecting columns

Filter and aggregation nodes with interactive editing

Clean, intuitive visual relationships

⚡ Live SQL Editor

Monaco Editor (VS Code engine) with SQL syntax highlighting

Real-time bidirectional sync between visual canvas and SQL

Import/export functionality for SQL files

Auto-completion and error detection

Fullscreen editing mode

📈 Instant Query Preview

Auto-refreshing results with 1-second debounce

Table view with sorting, searching, and pagination

Multiple chart types (bar, line, pie, scatter, area)

Export functionality (CSV download)

Performance metrics display

🎨 Design & User Experience

Databricks Brand Compliance: Applied official Databricks colors and design principles

Responsive Layout: Works seamlessly from desktop to smaller browser windows

Accessibility: ARIA labels, keyboard navigation, screen reader support

Modern UI: Clean, token-based styling with Tailwind CSS

Loading States: Comprehensive loading indicators and error handling

🏗️ Architecture & Technical Implementation

React 18 + TypeScript: Type-safe, modern React development

Modular Architecture: Clean separation between UI, data, and business logic

Context Management: Centralized state management with React Context

API Integration: Mock Databricks API integration ready for real implementation

Performance Optimized: Lazy loading, caching, and debounced operations

🚀 Deployment Ready

Databricks App Configuration: Complete app.yaml with proper permissions

Production Server: Express.js server with proxy middleware

Environment Configuration: Comprehensive environment variable setup

Build System: Vite-based build system optimized for production

Documentation: Extensive README with setup, development, and deployment guides

🔧 Key Features Implemented

✅ Auto-refresh background queries with loading indicators

✅ Databricks brand design system throughout the application

✅ SQL persistence - paste SQL to visualize, copy SQL for dashboards/jobs

✅ Drag-and-drop affordances with visual feedback

✅ Unity Catalog permissions properly configured

✅ Error handling with user-friendly messages

✅ Performance insights with execution time tracking

✅ Modular component architecture for maintainability

🚀 Getting Started

To run your new Visual SQL Query Builder:

Install dependencies:

bash

```
cd /Users/franco.patano/cursor\ projects/visualquerybuilder
npm install
```

Configure environment:

Create a .env file with your Databricks credentials:

env

```
DATABRICKS_HOST=https://your-workspace.cloud.databricks.com
DATABRICKS_TOKEN=your-access-token
DATABRICKS_WAREHOUSE_ID=your-warehouse-id
DATABRICKS_APP_PORT=3000
```

Start development:

bash
  npm run dev
Build for production:
bash
  npm run build
  npm start
The application is now ready to be deployed as a Databricks App and provides a complete visual SQL query building experience with all the requested features and modern design principles!

You are correct—your project from Cursor is a **React/TypeScript application**, not a Python/Streamlit one!
That means you need to use **Node.js and npm** (not Python, pip, or venv) to install packages, run, and build your app.

Here's What To Do Next

1. Verify You Have Node.js + npm Installed

- Open your terminal and run:

```
node -v
npm -v
```

You should see version numbers for both.
If not, install Node.js from nodejs.org. Recommended: Node.js LTS version.

2. Install NPM Dependencies

- In your project directory:

```
cd '/Users/franco.patano/cursor projects/visualquerybuilder'
npm install
```

This will read your `package.json` and install all required JavaScript dependencies into a `node_modules/` folder (ignore `requirements.txt` and pip for this app).

3. Set Up Environment Variables (.env File)

- In the same directory, create a file called `.env` (if it doesn't already exist).
- Add the following with your real values:

```
DATABRICKS_HOST=https://your-workspace.cloud.databricks.com
DATABRICKS_TOKEN=your-access-token
DATABRICKS_WAREHOUSE_ID=your-warehouse-id
DATABRICKS_APP_PORT=3000
```

4. Run the App in Development Mode

- Start the local dev server:

```
npm run dev
```

- This should launch your React app on `localhost:3000` (or whatever port is set).

5. (Optional) Build for Production

Once you're ready to deploy:

```
npm run build
npm start
```

6. Ignore Python venv for This Project

All Python/venv/pip setup is unnecessary for this React-based Databricks App.

Summary

- **Use `npm install`, not `pip install`.**
- **Set your env variables in a `.env` file in your project directory.**
- **Run the app with `npm run dev`.**

If you need to connect to Databricks APIs, make sure your access token is correct and that your warehouse is available.

If you want to remove the venv: Just delete the `venv/` folder.
If you have Python and JS code in the same repo, keep them separated.

Let me know if you see any errors when running npm install or npm run dev! I can troubleshoot Node or TypeScript issues as well.

(venv) (base) franco.patano@NL9VV2GC9H visualquerybuilder % node -v

npm -v

zsh: command not found: node
zsh: command not found: npm
(venv) (base) franco.patano@NL9VV2GC9H visualquerybuilder %

You do **not** have Node.js or npm installed, which is why you're seeing:

```
zsh: command not found: node
zsh: command not found: npm
```

Node.js is required for running React/TypeScript projects (like the one Cursor provided for your Visual Query Builder).

Here's How to Fix This

1. Download and Install Node.js

- Go to: https://nodejs.org/
- Click "LTS" (Recommended For Most Users) and download the macOS Installer.
- Run the downloaded `.pkg` file and follow the instructions.

  This will install both **node** and **npm** globally on your system.

2. Open a New Terminal Window

- After installation, fully close and reopen your terminal (or start a new tab/window).
- Run:

```
node -v
npm -v
```

You should now see version numbers like `v20.x.x` and `10.x.x` (versions may vary).

3. Continue With Your JavaScript Project

- Navigate back to your project:

  ```
  cd '/Users/franco.patano/cursor projects/visualquerybuilder'
  ```

- Then run:

  ```
  npm install
  ```

- Follow with:

  ```
  npm run dev
  ```

Or continue with further setup steps as needed.

You do NOT need Python or venv for this app. All work is now via Node.js/npm.

If you have any trouble installing Node.js or want to use a version manager (like nvm) for multiple versions, I can provide those steps as well. Let me know when you've installed Node.js and npm or if you get stuck!

---

do i need to configure the environment variables

Yes, you do need to configure the environment variables for your Databricks App to run correctly—especially since it needs to connect to your Databricks workspace and resources. Without these, your app won't be able to authenticate, fetch metadata, or execute queries.

Here's What You Need To Do

1. **Create a .env file** in your project directory (`/Users/franco.patano/cursor projects/visualquerybuilder`).

2. **Add these lines to your `.env` file** (replace the placeholder values with your actual Databricks details):

```
DATABRICKS_HOST=https://your-workspace.cloud.databricks.com
DATABRICKS_TOKEN=your-databricks-access-token
DATABRICKS_WAREHOUSE_ID=your-sql-warehouse-id
DATABRICKS_APP_PORT=3000
```

   - `DATABRICKS_HOST` – The URL of your Databricks workspace (e.g., `https://dbc-xxxx.cloud.databricks.com`)
   - `DATABRICKS_TOKEN` – A Personal Access Token from your Databricks account (User Settings > Developer > Generate new token)
   - `DATABRICKS_WAREHOUSE_ID` – The **ID** of your SQL Warehouse (can be found in the Databricks SQL workspace under SQL Warehouses)
   - `DATABRICKS_APP_PORT` – (For local dev) which port to run the app on. `3000` is common and works in most cases.

3. **Save the `.env` file**—the project's JavaScript/Node backend will automatically use it to configure your app.

Why Is This Necessary?

- These environment variables provide your app with secure and dynamic access to Databricks REST APIs and compute resources.
- **Without them**, requests for authentication, metadata, and query execution will fail.

Summary

- **Yes**, you must configure the environment variables.
- Do it **before running `npm run dev`** so your app connects successfully.

Let me know if you need help finding your warehouse ID or generating a Databricks token!

---

1. https://www.dbvis.com/thetable/what-is-an-sql-query-builder-and-how-does-it-work/
2. https://www.prisma.io/dataguide/types/relational/comparing-sql-query-builders-and-orms

3. https://docs.oracle.com/cd/E37097_01/doc.42/e35128/GUID-2CC5FE0D-1961-442E-9BD5-FA2056458B41.htm

4. https://docs.devart.com/studio-for-sql-server/building-queries-with-query-builder/query-builder-overview.html

5. https://mechanicalrabbit.github.io/FunSQL.jl/stable/two-kinds-of-sql-query-builders/

6. https://www.comparitech.com/net-admin/best-sql-query-builders/

7. https://www.thoughtspot.com/data-trends/data-modeling/best-sql-query-builders

8. https://www.metabase.com/features/query-builder

9. https://pnrsolution.org/Datacenter/Vol3/Issue3/169.pdf

10. http://dbgroup.eecs.umich.edu/files/SIGMOD14LFb.pdf

11. https://datavis.cs.columbia.edu/files/papers/nlpdb.pdf

12. http://vis.stanford.edu/files/2008-GeneralizedSelection-CHI.pdf

13. https://mode.com/blog/best-sql-query-builders/

14. https://support.k2view.com/Academy/articles/11_query_builder/01_query_builder_overview.html

15. https://www.reddit.com/r/SQL/comments/125jdbv/does_anyone_use_a_visual_sql_query_builder/

16. https://www.ibm.com/docs/en/data-studio/4.1.1?topic=builder-sql-query

17. https://help.karbonhq.com/en/articles/6350612-introduction-to-sql-for-the-query-builder

18. https://kafka.apache.org/10/documentation/streams/developer-guide/interactive-queries.html

19. https://learn.microsoft.com/en-us/fabric/data-warehouse/visual-query-editor

20. https://openhms.sourceforge.io/sqlbuilder/apidocs/overview-summary.html

21. https://docs.databricks.com/aws/en/sql/user/visualizations/

22. https://learn.microsoft.com/en-us/azure/databricks/visualizations/

23. https://docs.databricks.com/aws/en/sql/user/sql-editor/

24. https://learn.microsoft.com/en-us/azure/databricks/sql/user/sql-editor/

25. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/create-custom-app
26. https://docs.databricks.com/aws/en/dev-tools/databricks-apps
27. https://learn.microsoft.com/en-us/azure/databricks/generative-ai/agent-framework/chat-app
28. https://www.youtube.com/watch?v=VXkuVQLSvzk
29. https://www.cdata.com/kb/tech/databricks-odbc-active-query-builder.rst
30. https://grafana.com/whats-new/2024-02-06-databricks-sql-query-builder/
31. https://docs.databricks.com/aws/en/generative-ai/agent-framework/chat-app
32. https://docs.databricks.com/aws/en/sql/get-started/visualize-data-tutorial
33. https://docs.azure.cn/en-us/databricks/sql/user/sql-editor/
34. https://www.databricks.com/blog/2023/03/29/run-sql-queries-databricks-visual-studio-code.html
35. https://docs.databricks.com/aws/en/visualizations/
36. https://www.youtube.com/watch?v=tThmRuyp2cA
37. https://learn.microsoft.com/en-us/azure/databricks/sql/get-started/visualize-data-tutorial
38. https://docs.databricks.com/aws/en/dev-tools/sqltools-driver
39. https://docs.databricks.com/aws/en/getting-started/quick-start
40. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/sqltools-driver
41. https://www.linkedin.com/pulse/sql-query-builders-market-size-share-trends-outlook-zn42f
42. https://sites.google.com/view/trending-market-searches/market-research-reports/sql-query-builders-market
43. https://www.reddit.com/r/node/comments/173n3jp/we_migrated_to_sql_our_biggest_learning_dont_use/
44. https://stackoverflow.com/questions/2506610/visual-query-builder
45. https://www.toucantoco.com/en/blog/visual-sql-query-builder
46. https://ai2sql.io/learn/sql-for-social-media-analytics
47. https://www.sqlai.ai

48. https://dev.to/aleksikauppila/comment/hd03

49. https://forum.rescript-lang.org/t/ann-sql-query-builder/4620

50. https://mode.com/blog/best-sql-query-builders/

51. https://www.geeksforgeeks.org/sql/how-to-use-sql-for-social-media-data-analysis/

52. https://devrix.com/tutorial/social-media-sql/

53. https://blog.panoply.io/instagram-analytics-with-sql-queries-part-1-getting-started

54. https://forums.oracle.com/ords/apexds/post/oracle-sql-developer-query-builder-7368

55. https://stackoverflow.com/questions/63178744/sql-implementation-social-media-like-functionality

56. https://dl.acm.org/doi/10.1145/3613904.3642596

57. https://highscalability.com/stackoverflow-update-560m-pageviews-a-month-25-servers-and-i/

58. https://www.braze.com/docs/user_guide/analytics/query_builder/building_queries/

59. https://forums.allroundautomations.com/ubb/ubbthreads.php?ubb=showflat&Number=54181

60. https://news.ycombinator.com/item?id=42633880

61. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/app-development

62. https://www.tredence.com/blog/mastering-databricks-apps-from-creation-to-deployment-of-interactive-data-applications

63. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/databricks-apps/best-practices

64. https://learn.microsoft.com/en-us/azure/databricks/release-notes/product/2025/may

65. https://apps-cookbook.dev/blog/automate-apps-deployments-dabs/

66. https://softwaremill.com/pros-and-cons-of-using-streamlit-for-simple-demo-apps/

67. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/create-custom-app

68. https://www.linkedin.com/pulse/devops-best-practices-databricks-apps-neelabh-kashyap-olcjf

69. https://www.databricks.com/blog/introducing-databricks-one

70. https://www.reddit.com/r/databricks/comments/1jqbyv8/apps_or_ui_in_databricks/

71. https://anvil.works/articles/4-alternatives-streamlit

72. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/get-started

73. https://www.databricks.com/resources/demos/videos/databricks-apps-introduction

74. https://www.youtube.com/watch?v=Y5NU7F0tXh4

75. https://www.databricks.com/blog/introducing-databricks-apps

76. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/databricks-apps/app-development

77. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/best-practices

78. https://docs.databricks.com/aws/en/release-notes/product/2025/may

79. https://www.databricks.com/blog/building-databricks-apps-react-and-mosaic-ai-agents-enterprise-chat-solutions

80. https://docs.kanaries.net/topics/Streamlit/streamlit-vs-dash

81. https://anvil.works/articles/4-alternatives-streamlit

82. https://www.tredence.com/blog/mastering-databricks-apps-from-creation-to-deployment-of-interactive-data-applications

83. https://www.databricks.com/blog/building-databricks-apps-react-and-mosaic-ai-agents-enterprise-chat-solutions

84. https://www.linkedin.com/pulse/building-data-applications-databricks-apps-ivan-trusov-6pjwf

85. https://www.databricks.com/blog/introducing-databricks-apps

86. https://docs.databricks.com/aws/en/optimizations/disk-cache

87. https://overcast.blog/13-ways-to-optimize-databricks-queries-ce67f983b31b

88. https://www.databricks.com/resources/demos/videos/lakehouse-platform/data-lineage-with-unity-catalog

89. https://docs.databricks.com/aws/en/compute/troubleshooting/query-watchdog

90. https://eduardpopa.substack.com/p/the-ultimate-guide-to-cicd-for-data

91. https://azure-data.hashnode.dev/a-step-by-step-guide-to-cicd-using-databricks-asset-bundles-and-the-nutter-framework

92. https://www.fullstack.com/labs/resources/blog/accessibility-in-focus-the-intersection-of-screen-readers-keyboards-and-qa-testing

93. https://www.reddit.com/r/Python/comments/1149swv/company_project_djangoreact_streamlit_or_nonweb/

94. https://stackoverflow.com/questions/79080228/how-to-deploy-a-streamlit-app-using-databricks-app

95. https://learn.microsoft.com/en-us/azure/databricks/notebooks/notebook-editor

96. https://dynamicbusiness.com/topics/news/heres-how-you-can-build-data-apps-with-drag-and-drop-simplicity.html

97. https://www.databricks.com/discover/pages/data-quality-management

98. https://learn.microsoft.com/en-us/answers/questions/1684230/results-too-large-error

99. https://stackoverflow.com/questions/77216860/launch-streamlit-application-under-azure-databricks

100. https://www.databricks.com/blog/announcing-general-availability-databricks-apps

101. https://docs.databricks.com/aws/en/sql/user/sql-editor/

102. https://docs.databricks.com/aws/en/visualizations/

103. https://docs.databricks.com/aws/en/sql/user/visualizations/

104. https://www.databricks.com/product/databricks-apps

105. https://grafana.com/whats-new/2024-02-06-databricks-sql-query-builder/

106. https://github.com/mullerpeter/databricks-grafana

107. https://www.cdata.com/kb/tech/databricks-odbc-active-query-builder.rst

108. https://github.com/dbeaver/dbeaver/wiki/Visual-Query-Builder

109. https://www.appsmith.com/integration/databricks

110. https://community.alteryx.com/t5/Alteryx-Designer-Desktop-Discussions/Databricks-Unable-to-see-all-schemas-and-tables/td-p/1329064

111. https://www.reddit.com/r/databricks/comments/1ktfz7b/dbx_compatible_query_builder_for_a_typescript/

112. https://docs.databricks.com/aws/en/sql/get-started/visualize-data-tutorial

113. https://azuremarketplace.microsoft.com/en-us/marketplace/apps/cozyrocllc1726745399505.databricks?tab=overview

114. https://www.databricks.com/blog/2023/03/29/run-sql-queries-databricks-visual-studio-code.html

115. https://github.com/dbeaver/dbeaver/wiki/Visual-Query-Builder/c4a38455c978f9ea2ac303b82256734d7869df18

116. https://docs.databricks.com/aws/en/dev-tools/sql-drivers-tools

117. https://www.reddit.com/r/databricks/comments/1g6u0u5/creating_sql_table_in_databrick_community/

118. https://www.databricks.com/product/open-source

119. https://docs.azure.cn/en-us/databricks/sql/user/sql-editor/

120. https://www.reddit.com/r/databricks/comments/1dh3vv9/most_successful_bi_analytics_tool_with_databricks/

121. https://docs.databricks.com/aws/en/dev-tools/auth/

122. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/auth

123. https://registry.terraform.io/providers/databricks/databricks/latest/docs

124. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/databricks-apps/auth

125. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/key-concepts

126. https://docs.databricks.com/aws/en/dev-tools/bundles/variables

127. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/bundles/settings

128. https://www.cloudthat.com/resources/blog/building-smarter-workflows-with-databricks-dynamic-value-references/

129. https://docs.databricks.com/aws/en/jobs/parameter-use

130. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/environment-variables

131. https://learn.microsoft.com/en-us/azure/databricks/dev-tools/auth/

132. https://docs.databricks.com/aws/en/dev-tools/auth/pat

133. https://databricks-sdk-py.readthedocs.io/en/latest/authentication.html

134. https://docs.databricks.com/aws/en/security/auth/access-control

135. https://www.getorchestra.io/guides/how-to-retrieve-your-databricks-warehouse-id

136. https://stackoverflow.com/questions/73810000/how-can-we-get-the-databricks-workspace-url-and-access-token

137. https://docs.databricks.com/aws/en/workspace/workspace-details

138. https://servian.dev/dynamic-sql-in-databricks-notebooks-using-sql-7fe9aca4d767

139. https://docs.databricks.com/aws/en/dev-tools/databricks-apps/permissions

140. https://docs.databricks.com/aws/en/dev-tools/cli/reference/warehouses-commands